Chapter 5: Using Structs to structure related data

A struct, or structure, is a custom data type that lets you name and package together multiple related values that make up a meaningful group.
If you're familiar with an object-oriented language, a struct is like an object's data attributes.

Defining and Instantiating Struct

To define a struct, we enter the keyword struct and name the entire struct.
A struct's name should describe the significance of the pieces of data being grouped together.
Then, inside curly brackets, we define the names and types of the pieces of data, which we call fields.

```
struct User {
    username: String,
    email: String,
    sign_in_count: u64,
    active: bool,
}
```

To use a struct after we've defined it, we create an instance of that struct by specifying concrete values for each of the fields.
We create an instance by stating the name of the struct and then add curly brackets containing key: value pairs, where the keys are the names of the fields and the values are the data we want to store in those fields.

```
let user1 = User {
    email: String::from("someone@example.com"),
    username: String::from("someusername123"),
    active: true,
    sign_in_count: 1,
};
```

To get a specific value from a struct, we can use dot notation.
If we wanted just this user's email address, we could use 'user1.email 'wherever we wanted to use this value.
If the instance is mutable, we can change a value by using the dot notation and assigning into a particular field.

```
let mut user1 = User {
    email: String::from("someone@example.com"),
    username: String::from("someusername123"),
    active: true,
    sign_in_count: 1,
};

user1.email = String::from("anotheremail@example.com");
```

Note that the entire instance must be mutable;
Rust doesn't allow us to mark only certain fields as mutable.

As with any expression, we can construct a new instance of the struct as the last expression in the function body to implicitly return that new instance.

For example, here we have 'build_user()' function that returns a 'User' instance with the given email and username.

The ‘active’ field gets a value of ‘true’, and the ‘sign_in_count’ gets a value of ‘1’.

```
fn build_user(email: String, username: String) -> User {
    User {
        email: email,
        username: username,
        active: true,
        sign_in_count: 1,
    }
}
```

Field Init Shorthand (when variables and fields names are matching)

Because we choose the same names for parameters and fields in the previous example, we can use the field init shorthand syntax to rewrite ‘build_user()’ so that it behaves exactly the same but doesn’t have the repetition of ‘email’ and’username’:

```
fn build_user(email: String, username: String) -> User {
    User {
        email,
        username,
        active: true,
        sign_in_count: 1,
    }
}
```

Creating instances from other instances with Struct Update Syntax

We define ‘user2’ as follows:

```
let user2 = User {
    email: String::from("another@example.com"),
    username: String::from("anotherusername567"),
    active: user1.active,
    sign_in_count: user1.sign_in_count,
};
```

Using struct update syntax, we can achieve the same effect with less code, as shown:

```
let user2 = User {
    email: String::from("another@example.com"),
    username: String::from("anotherusername567"),
    ..user1
};
```

The syntax ‘…’ specifies that the remaining fields not explicitly set should have the same value as the fields in the given instance.

Unit-Like structs without any fields

These are called unit-like structs because they behave similarly to (), the unit type.
Unit-like structs can be useful in situations in which you need to implement a trait on some type but don’t have any data that you want to store in the type itself.

Adding useful functionality: Struct Debug

We've now developed our 'rectangle/' program which allows us to showcase how structs are implemented.
Let's now say that we want to see the values for all fields of a Rectangle instance (eg. 'rect1');
We want to define that the struct is being debugged by adding:

```
#[derive(Debug)]
struct Rectangle {
    width: u32,
    height: u32,
}
```

And use ':?' or ':#?' inside curly brackets of a 'println!' to display the values as follows:

```
println!("rect1 is {:?}", rect1);
```

Method syntax

Methods are different from functions in that they're defined within the con- text of a struct, and their first parameter is always self, which represents the instance of the struct the method is being called on.

Defining methods

To define the function within the context of Rectangle, we start an 'impl' block. Then we move the 'area' function within the 'impl' curly brackets and change the first parameter to be 'self' in the signature and everywhere within the body.

In main, where we called the area function and passed rect1 as an argument, we can instead use method syntax to call the area method on our Rectangle instance.
The method syntax goes after an instance: we add a dot followed by the method name, parentheses, and any arguments.

Methods can take ownership of 'self', borrow 'self' immutably as we've done here, or borrow 'self' mutably, just as they can any other parameter.

We don't want to take ownership, and we just want to read the data in the struct, not write to it (therefore we used '&self').
If we wanted to change the instance that we've called the method on as part of what the method does, we'd use '&mut self' as the first parameter.

Rust interpretation of '->' operator

Rust doesn't have an equivalent to the '->' operator; instead, Rust has a fea- ture called automatic referencing and dereferencing.
Calling methods is one of the few places in Rust that has this behavior. Here's how it works: when you call a method with 'object.something()', Rust automatically adds in '&', '&mut', or '*' so object matches the signature of the method.

In other words, the following are the same:

```
p1.distance(&p2);
(&p1).distance(&p2);
```

Associated Functions ()

Another useful feature of impl blocks is that we're allowed to define functions within impl blocks that don't take

self as a parameter.
These are called associated functions because they're associated with the struct.

Associated functions are often used for constructors that will return a new instance of the struct.
Here is an example:

```
impl Rectangle {
    fn square(size: u32) -> Rectangle {
        Rectangle { width: size, height: size }
    }
}
```

To call this associated function, we use the '::' syntax with the struct name; 'let sq = Rectangle::square(3);' is an example.